

Assignment 4 - Advanced Programming 2026

Anton Bang Lejbølle & Jonas Dennis Hansen

October 2026

1 Introduction

For this assignment, we do not have any general concerns or have found any ambiguities in the problem text. We are confident that our solution is of high quality as it is functional and passes all the test cases we have written for it. There might still be some edge cases that we have not considered, but we believe that our solution is robust and handles the majority of scenarios effectively.

We have implemented try-catch effects, key-value store effects, a transactional computation effect and breaking out of loops in both a pure interpreter and a IO-based interpreter. In the IO-based interpreter we have also implemented an interactive missing key recovery. For both interpreters we have written various test cases to cover its functionality. These test cases include all the examples given in assignment and a number of relevant cases to cover more of the specification given in the assignment. The test cases together cover that TryCatch catches failure, TryCatch keeps successful result, TryCatch propagates fallback failure, KvGet gets the value given by KvPut, KvPut overwrites existing key, KvGet fails when missing key in the pure interpreter, KvGet prompts when missing key in the IO-based interpreter, Transaction commits successful changes, failed transaction rolls back changes, break stops loop, break out of loop fails, and double break still stops loop.

2 Answers to questions

1. Consider interpreting a TryCatchOp m1 m2 k effect where m1 fails after performing some key-value store effects.

(a)

Yes, there is a difference between our pure interpreter and our IO-based interpreter in terms of whether the key value store effects that m1 performed before it failed are visible when interpreting m2.

In our pure interpreter, key-value store effects that m1 performed before it failed **are not** visible when interpreting m2

In our IO-based interpreter the key-value store effects that m1 performed before it failed **are** visible when interpreting m2.

This difference is because in our pure interpreter, if m1 fails then the state that stores the performed key-value store effects is lost, but in our IO-based interpreter, if m1 fails then the performed key-value store effects have already been written to the db, so these are not lost.

(b)

Yes, if we had implemented both our interpreters such that the key-value store effects that m1 performed before it failed are always visible when interpreting m2, you could just use the transaction operator in the beginning of m1. This would make all the key-value store effects in m1 to be invisible in m2.

2. Why is TransactionOp not defined as follows?

TransactionOp is not defined as "TransactionOp (EvalM ()) a", because then it does not have an EvalM Val payload. This would mean that TransactionOp could only execute computations returning the unit (), and would therefore discard the value produced by the expression.

3. Why is TryCatchOp not defined as follows?

TryCatchOp is not defined as "TryCatchOp a a", because a is the result of a computation. We are therefore never in a situation, where you can catch it, as it is evaluated after the computation has taken place. Therefore with TryCatchOp a a, the computations would already have been evaluated, so TryCatch would no longer be able to control when they are executed or catch their failures.